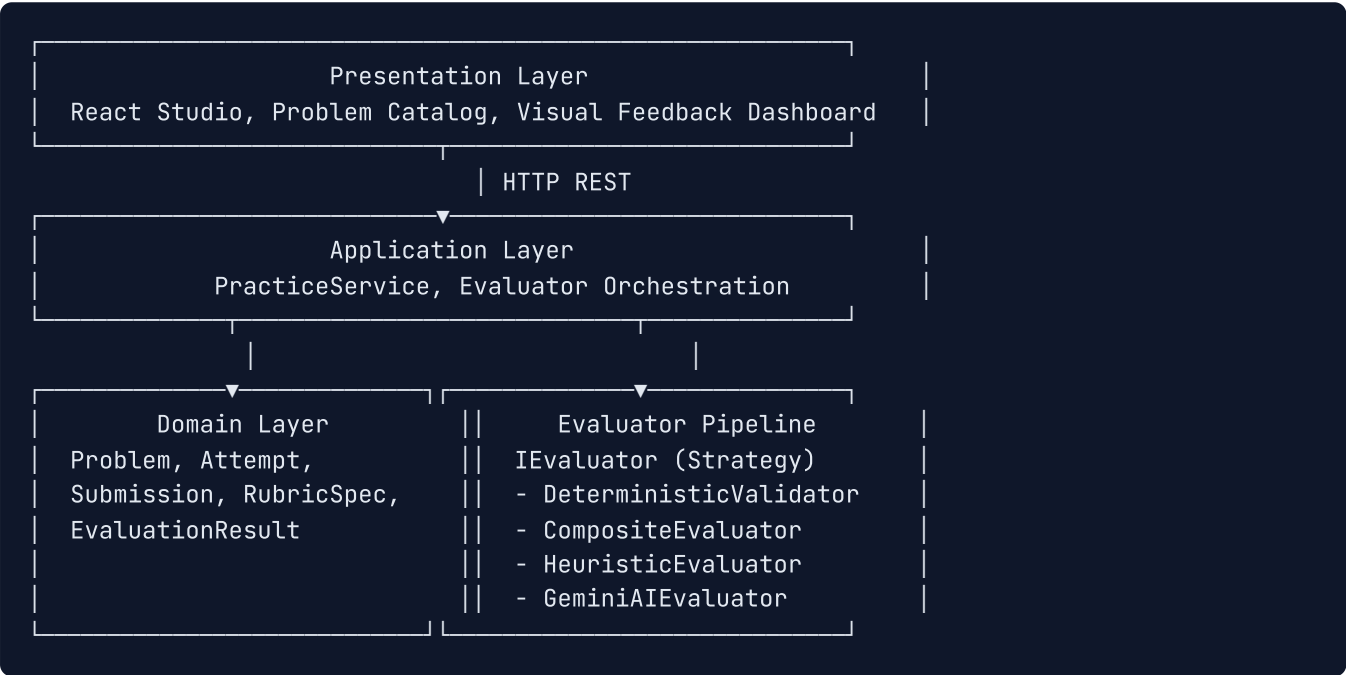


Design Note: LLD Practice Platform Architecture & Domain Model

Platform: DesignForge LLD Practice Studio | **Focus:** Clean Architecture, Domain Models, Evaluator Strategy & Trade-offs | **Date:** September 2026

1. System Architecture & Domain Model

The platform itself is engineered following strict Clean Architecture and Domain-Driven Design (DDD) principles:



Core Domain Entities & Responsibilities

Class / Interface	Primary Responsibility	Collaborators / Dependencies
Problem	Encapsulates challenge metadata, functional/non-functional requirements, constraints, expected entities, and rubric specifications.	RubricCriterionSpec , StarterTemplate
Attempt	Aggregate Root managing the lifecycle of a learner's attempts (IN_PROGRESS → EVALUATING → EVALUATED → FAILED).	Submission[] , Problem
Submission	Entity representing a specific version snapshot (\$v1, v2, ...\$) containing Assumptions, Mermaid diagram, code, language, and design rationale.	SubmissionContent , EvaluationResult

Class / Interface	Primary Responsibility	Collaborators / Dependencies
<code>IEvaluator</code>	Polymorphic Strategy interface defining the evaluation contract (<code>evaluate(problem, submission)</code>).	<code>Problem</code> , <code>SubmissionContent</code> , <code>EvaluationResult</code>
<code>DeterministicValidator</code>	Performs AST static checks: verifies mandatory sections, extracts classes/interfaces/enums, detects God classes and public state leaks.	<code>CodeStructureAnalysis</code> , <code>DeterministicCheckResult</code>
<code>HeuristicEvaluator</code>	Zero-baseline rule engine calculating 5–dimension rubric scores, verbatim evidence quotes, concerns, and actionable advice offline.	<code>DeterministicValidator</code> , <code>RubricCriterionSpec</code>
<code>GeminiAIEvaluator</code>	LLM-based evaluator leveraging Google Gemini with strict JSON schema prompt; falls back gracefully to Heuristic engine.	<code>IEvaluator</code> , <code>DeterministicValidator</code>
<code>CompositeEvaluator</code>	Orchestrator combining deterministic sanity checks with AI / Heuristic evaluation strategies.	<code>IEvaluator</code> , <code>DeterministicValidator</code>
<code>PracticeService</code>	Application Service coordinating problem retrieval, attempt state transitions, submission evaluation, and version delta comparisons.	<code>ProblemRepository</code> , <code>AttemptRepository</code> , <code>IEvaluator</code>

2. Extensibility & Change Tests

The platform was subjected to two rigorous change tests to validate adherence to SOLID principles:

- Change Test A:** *"Today the learner submits text. Later the platform supports an interactive drag-and-drop UML canvas."*
Domain Impact: Zero. `SubmissionContent` is an encapsulated value object containing `diagramMermaid: string`. Introducing a visual canvas only extends the serializer in the presentation layer and adds an optional schema field to `SubmissionContent`. `Attempt` , `Problem` , and `PracticeService` remain untouched.
- Change Test B:** *"Today feedback comes from one evaluator. Later you add a peer review, automated unit test runner, or human mentor review."*
Domain Impact: Zero. `IEvaluator` is a polymorphic strategy interface. Adding `HumanMentorEvaluator` or `AutomatedUnitTestRunner` only requires implementing `IEvaluator.evaluate()` and registering it in `CompositeEvaluator` (Open-Closed Principle).

3. Key Trade-offs Made for the MVP

- Monolithic In-Memory Repository vs. Distributed Database:** Clean repository pattern defaults to fast durable in-memory storage while supporting PostgreSQL via `DATABASE_URL` with zero code changes.

2. **Hybrid Heuristic Engine vs Pure Cloud LLM:** Rule-based `HeuristicEvaluator` guarantees 100% offline availability and instant execution if no `GEMINI_API_KEY` is provided.
3. **Structured Tabbed Editor vs Heavy Container Sandbox:** Focused on architectural decomposition and OOP contracts rather than heavy Docker container compilation overhead.